State-of-the-art : les algorithmes des jeux de stratégie au tour par tour de type Civilization

Maxence ARDITTI · Armel Stephane Novak KI · Ababacar DIENE · Wassim OUKAZI

Introduction

Les jeux de stratégie au tour par tour de type **4X** (eXplore, eXpand, eXploit, eXterminate) constituent un genre à part entière, popularisé par la série *Civilization* de Sid Meier depuis 1991. En plus de trente ans, la série a vendu plus de 60 millions d'exemplaires et a inspiré des dizaines de jeux dérivés (*Freeciv*, *OpenCiv*, *Endless Legend*, *Humankind*).

Derrière leur apparente simplicité, ces jeux mobilisent un ensemble d'algorithmes variés : génération procédurale de carte, recherche de chemin optimal, intelligence artificielle des entités ennemies, gestion des ressources, visibilité partielle. Chacun de ces modules pose des problèmes algorithmiques distincts, avec leurs contraintes de complexité et leurs compromis.

L'objectif de ce rapport est de présenter les algorithmes classiques et modernes utilisés dans ce type de jeu, en distinguant les approches applicables dans un contexte de développement étudiant des approches industrielles. Nous nous appuierons sur notre propre implémentation en C du projet *Les Incivilisés*, un clone textuel de Civilization développé dans le cadre du cours PPII.

Sommaire

- Génération procédurale de carte
  - Le bruit de Perlin
  - L'automate cellulaire
  - Les diagrammes de Voronoï
- Pathfinding : trouver le chemin optimal
  - BFS (Breadth-First Search)
  - L'algorithme de Dijkstra
  - L'algorithme A*
- Intelligence artificielle des entités
  - Les machines à états finis (FSM)
  - Les arbres de comportement (Behavior Trees)
  - Monte Carlo Tree Search (MCTS)
- Gestion des ressources et de l'économie
- Le brouillard de guerre (Fog of War)
- Arbres technologiques et graphes de dépendances
- Conclusion

Génération procédurale de carte

La génération procédurale consiste à créer du contenu de jeu algorithmiquement plutôt que manuellement. Pour un jeu type Civilization, cela concerne en premier lieu la carte du monde : elle doit être jouable (pas trop d'eau, des ressources accessibles), visuellement cohérente (les montagnes forment des chaînes, les forêts se regroupent) et différente à chaque partie.

Le bruit de Perlin

Le bruit de Perlin, inventé par Ken Perlin en 1983 pour le film *Tron*, est l'algorithme de référence pour générer des terrains naturels. À partir d'une grille de vecteurs de gradient pseudo-aléatoires, il produit pour chaque coordonnée `(x, y)` une valeur continue entre -1 et 1 avec une propriété fondamentale : **les valeurs proches dans l'espace sont proches en valeur** (continuité de Hölder), ce qui donne un résultat organique à l'opposé d'un bruit blanc.

En pratique, on superpose plusieurs **octaves** du bruit — chaque octave ayant une fréquence deux fois plus élevée et une amplitude deux fois plus faible — pour obtenir des détails à plusieurs échelles :

- 1er octave → formes des continents et mers
- 2e octave → reliefs régionaux (collines, vallées)
- 3e octave → variations locales de texture

Le seuillage de la valeur finale détermine le type de terrain :

Valeur normalisée	Type de terrain
< 0.3	Eau profonde
0.3 – 0.45	Côte / eau peu profonde
0.45 – 0.6	Plaine

Valeur normalisée	Type de terrain
0.6 – 0.75	Forêt
0.75 – 0.9	Colline
> 0.9	Montagne

Une variante, le **bruit Simplex** (également de Ken Perlin, 2001), réduit les artefacts visuels et améliore les performances en 3D. Il reste cependant plus complexe à implémenter.

L'automate cellulaire

L'automate cellulaire est une approche plus simple, souvent utilisée pour les niveaux de type donjon ou île. On part d'une grille initialisée aléatoirement (chaque case est "terre" avec une probabilité *p*, généralement 0.45) puis on applique des règles de voisinage de manière itérative :

> Si une case a **5 voisins ou plus** de type "terre" (voisinage de Moore 3×3), elle devient "terre" au prochain tour ; sinon, elle devient "eau".

Après 4 à 5 itérations, on obtient des masses terrestres continues et des îlots cohérents. Un post-traitement par **flood-fill** permet d'identifier et de supprimer les petites îles isolées, garantissant une carte jouable.

C'est moins réaliste que le bruit de Perlin mais **bien plus rapide à implémenter** et suffisant pour un prototype — c'est l'approche que nous aurions pu adopter pour *Les Incivilisés*.

Les diagrammes de Voronoï

Dans les jeux modernes comme *Civilization VI*, la carte est divisée en **tuiles hexagonales** dont les territoires de villes sont calculés par diagrammes de Voronoï : chaque case appartient à la ville la plus proche au sens d'une distance pondérée.

Le diagramme de Voronoï peut être calculé efficacement par l'algorithme de Fortune en O(n log n). En jeu, il permet de délimiter les frontières de manière naturelle et de calculer dynamiquement l'influence d'une ville sur son territoire.

Pathfinding : trouver le chemin optimal

Le pathfinding est au cœur de tout jeu de stratégie : comment une unité doit-elle se déplacer d'un point A à un point B sur une grille, en évitant les obstacles et en minimisant le coût de déplacement ?

BFS (Breadth-First Search)

La **recherche en largeur** explore les cases voisines couche par couche, en maintenant une file (FIFO). Elle garantit de trouver le chemin le plus court **en nombre de cases** sur une grille où tous les déplacements ont le même coût.

```
file = [départ]
parent[départ] = NULL
tant que file non vide :
    case = file.dépiler()
    si case == destination : reconstruire le chemin
    pour chaque voisin de case :
        si voisin non visité et non bloqué :
            parent[voisin] = case
            file.enfiler(voisin)
```

Avantage : simple à implémenter, garantit l'optimalité sur grille uniforme.
Inconvénient : explore dans toutes les directions sans tenir compte de la destination — coûteux sur de grandes cartes.
Complexité : O(V + E) où V est le nombre de cases et E le nombre d'arêtes.

Dans notre projet, le BFS est utilisé pour le déplacement des barbares : il leur permet de trouver la direction vers la cible la plus proche (ville ou unité) en un seul passage.

L'algorithme de Dijkstra

Dijkstra généralise le BFS au cas où les cases ont des **coûts différents**. Il maintient une file de priorité (min-heap) triée par coût cumulé depuis la source.

C'est l'algorithme adapté dès qu'on introduit des terrains avec des coûts de mouvement variables :

Terrain	Coût de déplacement
Plaine	1 PM
Forêt	2 PM
Colline	3 PM

Terrain	Coût de déplacement
Montagne	infranchissable
Eau	infranchissable (unités terrestres)

Dijkstra garantit le chemin de **coût total minimal**, même avec ces variations.
Complexité : O((V + E) log V) avec un tas binaire.

L'algorithme A*

A* est l'extension de Dijkstra qui intègre une **heuristique h(n)** estimant le coût restant jusqu'à la destination. La file de priorité trie non plus par coût réel g(n) mais par f(n) = g(n) + h(n).

Pour une grille 2D, les heuristiques courantes sont :

- **Distance de Manhattan** : $|x_1-x_2| + |y_1-y_2|$ (adaptée aux grilles sans diagonales)
- **Distance euclidienne** : $\sqrt{(x_1-x_2)^2 + (y_1-y_2)^2}$
- **Distance de Chebyshev** : $\max(|x_1-x_2|, |y_1-y_2|)$ (adaptée aux 8-voisins)

A* est **l'algorithme standard des jeux vidéo** pour le pathfinding sur grille. Il est significativement plus rapide que Dijkstra sur de grandes cartes car il ne développe pas les zones éloignées de la trajectoire optimale.

La qualité du résultat dépend du choix de l'heuristique : elle doit être **admissible** (ne jamais surestimer le coût réel) pour garantir l'optimalité. Si l'on accepte de sacrifier l'optimalité pour la vitesse, une heuristique pondérée f(n) = g(n) + w·h(n) avec w > 1 (*A pondéré** ou *Weighted A**) peut être jusqu'à 10× plus rapide au prix de chemins légèrement sous-optimaux.

Intelligence artificielle des entités

L'IA d'un jeu de stratégie gouverne le comportement des unités ennemies (barbares, civilisations adverses) et des entités neutres. Elle doit être crédible, réactive et — idéalement — imprévisible sans être injuste.

Les machines à états finis (FSM)

La **Finite State Machine** est l'approche la plus répandue en jeu vidéo. Une unité est dans un état à un instant donné et transite vers un autre état selon des conditions précises.

Exemple pour un barbare :

```
États : PATROUILLE → POURSUITE → ATTAQUE → FUITE

Transitions :
  PATROUILLE → POURSUITE  : ennemi détecté dans un rayon R
  POURSUITE  → ATTAQUE    : ennemi adjacent (distance ≤ 1)
  ATTAQUE    → FUITE      : HP < 20% du maximum
  FUITE      → PATROUILLE : ennemi hors de vue
```

La FSM est **explicite, déterministe et facile à déboguer**, ce qui en fait le choix naturel pour un projet étudiant. Sa limite principale est la **combinatoire** : avec beaucoup d'états et de transitions, le graphe devient difficile à maintenir et les interactions entre états peuvent créer des comportements inattendus.

Dans notre implémentation, les barbares utilisent une FSM simplifiée : ils attaquent s'ils sont adjacents à une cible, sinon ils se déplacent vers la cible la plus proche par BFS, sinon ils se déplacent aléatoirement.

Les arbres de comportement (Behavior Trees)

Les **Behavior Trees** (BT) sont une évolution des FSM, largement adoptés dans l'industrie (*Halo*, *Alien: Isolation*, *The Last of Us*). Ils structurent les décisions en arbre :

- **Nœud Séquence (→)** : exécute les enfants de gauche à droite ; échoue dès qu'un enfant échoue
- **Nœud Sélection (?)** : essaie les enfants de gauche à droite ; réussit dès qu'un enfant réussit
- **Feuilles** : conditions (est-ce qu'un ennemi est visible ?) ou actions (attaquer, se déplacer)

Leur avantage par rapport aux FSM est la **modularité** : on peut réutiliser des sous-arbres et composer des comportements complexes sans exploser en nombre de transitions. Un barbare "intelligent" pourrait ainsi avoir un comportement de groupe : rejoindre d'autres barbares proches avant d'attaquer.

Monte Carlo Tree Search (MCTS)

Le **MCTS** est l'algorithme qui a révolutionné les jeux de plateau complexes (Go, échecs avec AlphaGo/AlphaZero). Il évalue les décisions par simulation aléatoire selon quatre phases répétées :

1. **Sélection** : descendre dans l'arbre de décision en favorisant les nœuds peu explorés (formule UCB1)
2. **Expansion** : ajouter un nœud fils non encore exploré
3. **Simulation** : jouer une partie aléatoire (*rollout*) depuis ce nœud
4. **Rétropropagation** : mettre à jour les scores de tous les nœuds ancêtres

Après des milliers d'itérations, on garde la décision qui mène statistiquement aux meilleurs résultats. Dans Civilization, une IA basée sur MCTS pourrait décider quelle technologie rechercher, quelle ville attaquer ou quel accord diplomatique accepter en simulant les conséquences à plusieurs tours.

Son coût computationnel est élevé et la qualité du *rollout* (simulation aléatoire) dégrade les résultats si les décisions aléatoires sont trop éloignées du jeu réel. Dans

un contexte étudiant, les FSM ou les règles heuristiques restent plus adaptés.

Gestion des ressources et de l'économie

La gestion des ressources (or, production, science, nourriture) dans un jeu 4X suit des **modèles économiques discrets** : chaque tour, chaque ville génère des revenus selon ses bâtiments, sa population et son territoire.

Revenus et dépenses

Le modèle de base est une **somme pondérée** par tour :

```
or_joueur += Σ(or_ville_i) - entretien_unités - coût_bâtiments
science   += Σ(science_ville_i) + bonus_bibliothèques
production += production_ville_i + bonus_atelier
```

Dans notre projet, l'entretien des unités est calculé globalement en fin de tour (`process_player_gold`) : chaque unité au-delà d'un certain seuil coûte de l'or, et si le joueur ne peut pas payer, des unités sont automatiquement dissoutes.

L'arbre de production et les prérequis

La production d'unités ou de bâtiments suit un système de **prérequis** formant un graphe orienté acyclique (DAG). Certaines unités nécessitent une technologie débloquée ET un bâtiment en ville :

Unité / Bâtiment	Technologie requise	Bâtiment requis
Archer	Chasse	Caserne
Cavalier	Équitation	Caserne
Bibliothèque	Écriture	—
Remparts	Maçonnerie	—

Ce système garantit une **progression cohérente** : le joueur doit investir en science avant d'accéder aux unités avancées.

Équilibrage

L'équilibrage est un problème ouvert : dans *Civilization VI*, les développeurs ajustent encore les coûts de production et les statistiques des unités par patches réguliers, après des milliers d'heures de tests. Pour un projet étudiant, un équilibrage empirique (tests de parties complètes) est suffisant.

Le brouillard de guerre (Fog of War)

Le Fog of War est un mécanisme central des jeux de stratégie : il introduit une **asymétrie d'information** entre le joueur et la carte, forçant l'exploration active.

On distingue trois états pour chaque case :

État	Description
Non-découverte	Jamais visitée, invisible
Découverte (mémoire)	Visitée autrefois, terrain visible, unités non visibles
Visible	Dans le rayon de vision d'une unité

Calcul de la visibilité

L'approche naïve consiste à parcourir toutes les unités et à marquer les cases dans leur rayon. Pour une grille de taille raisonnable (≤ 100×100) et un nombre d'unités limité, cette approche O(unités × rayon²) est suffisante.

Pour les grandes cartes, des algorithmes de **shadow-casting** (projection d'ombres) permettent de calculer exactement les cases visibles depuis un point, en tenant compte des obstacles (montagnes bloquant la vue). L'algorithme de Bresenham est souvent utilisé pour tracer des lignes de vue discrètes sur grille.

Impact sur l'IA

Le Fog of War complique l'IA ennemie : une IA omnisciente (qui voit toute la carte) est injuste. Une IA réaliste ne devrait attaquer que les cibles qu'elle "voit". Dans la pratique, beaucoup de jeux accordent à l'IA une visibilité partielle ou complète pour compenser sa faiblesse stratégique — c'est un choix de design assumé.

Arbres technologiques et graphes de dépendances

L'arbre technologique est un **DAG** (Directed Acyclic Graph) : chaque technologie a des prérequis à débloquer avant d'être accessible. Dans *Civilization VI*, cet arbre comprend plus de 60 technologies avec des chemins multiples.

Représentation

La représentation la plus naturelle est une **matrice d'adjacence** ou une **liste d'adjacence** :

```
// Liste d'adjacence simple pour les prérequis
int prerequisites[MAX_TECHNOLOGIES][MAX_TECHNOLOGIES];
```

Une traversée en ordre topologique (DFS post-order) permet de vérifier si toutes les dépendances d'une technologie sont satisfaites avant de l'autoriser.

Algorithmes de recommandation

Dans les jeux commerciaux, l'IA doit choisir quelles technologies rechercher. Les approches vont des **règles heuristiques** simples (toujours rechercher ce qui débloque les unités militaires en priorité) à des approches de **planification par satisfaction de contraintes** (CSP) : trouver la séquence de technologies qui maximise un score à horizon N tours.

Conclusion

Les jeux de stratégie type Civilization sont un terrain d'application remarquable pour l'algorithmique : génération procédurale, pathfinding, IA comportementale, gestion de ressources et visibilité partielle coexistent dans un même système cohérent.

Pour un contexte de développement étudiant, les choix pragmatiques sont :

- **Bruit de Perlin** pour la carte (ou automate cellulaire pour plus de simplicité)
- **BFS** pour les déplacements simples, **A*** pour un pathfinding coût-optimal
- **FSM** pour l'IA des unités ennemies
- **DAG + DFS** pour l'arbre technologique

Ces algorithmes sont explicables, implémentables en C sans bibliothèques externes et suffisamment performants pour une grille de taille raisonnable (50×30 dans notre projet).

Les approches industrielles — MCTS pour la stratégie globale, Behavior Trees hiérarchiques, génération neuronale de cartes — offrent des résultats bien supérieurs mais au prix d'une complexité d'implémentation et d'un coût computationnel qui dépassent le cadre d'un projet académique. Elles constituent néanmoins des perspectives d'extension naturelles pour une version post-PPII du projet.

L'intelligence artificielle dans les jeux de stratégie est un domaine de recherche actif : des travaux récents explorent l'entraînement d'agents par **renforcement profond** (Deep RL) capables de battre des joueurs humains experts à Civilization IV et V, ouvrant la voie à des IA adversaires véritablement imprévisibles.

State-of-the-art rédigé dans le cadre du projet PPII-FISE — Les Incivilisés, 2025-2026.